

Java HashMap

The Complete Guide to Key-Value Architectures

CODEHIVE

CODEHIVE

1. Introduction to Hashing

A **HashMap** stores items in **Key/Value pairs**. Imagine a dictionary: the "Word" is the Key, and the "Definition" is the Value. It is part of the `java.util` package and is the go-to structure for high-speed data retrieval.

Key Philosophy:

- **Identity over Index:** Access elements using meaningful keys instead of arbitrary integers.
- **The Unique Key Rule:** Every key must be unique. If you add "USA" twice, the second entry overwrites the first.
- **Duplicate Values:** Multiple keys can point to the same value (e.g., "USA" → "DC", "UK" → "London").

Note: HashMap does not guarantee any specific order of elements.

2. Initialization

To use `HashMap`, you must import the class and specify the data types for both the **Key** and the **Value** using Generics.

```
import java.util.HashMap;

// Initializing a Map of String Keys and String Values
HashMap<String, String> capitalCities = new HashMap<>();
```

The Anatomy of a Declaration

In `HashMap<String, String>`:

- The first `String` is the **Key type**.
- The second `String` is the **Value type**.

3. The .put() Method

Adding items to a HashMap is simple using the `put ( )` method. This method takes two arguments: the Key and the Value.

```
capitalCities.put("England", "London");  
capitalCities.put("India", "New Delhi");  
capitalCities.put("Norway", "Oslo");  
capitalCities.put("USA", "Washington DC");
```

Deep Dive: Overwriting

If you execute `put ( "England", "Manchester" )` now, the value "London" is deleted and replaced by "Manchester". The Key "England" remains unique.

4. Accessing Information

To retrieve a value, you supply the key to the `get()` method. This is a constant-time operation—it's incredibly fast regardless of map size.

```
// Retrieves "London"  
String city = capitalCities.get("England");  
  
System.out.println("The capital is: " + city);
```

Size of the Collection

To find out how many pairs are currently stored, use the `size()` method.

```
int totalPairs = capitalCities.size(); // Returns 4
```

5. Removing Items

Java provides two main ways to remove data: targeting a specific entry or wiping the entire map clean.

```
// 1. Remove a specific entry by its key
capitalCities.remove("England");

// 2. Remove all entries at once
capitalCities.clear();
```

Best Practice: Always check if a key exists before performing complex logic using `containsKey()`. This prevents `NullPointerExceptions` when trying to manipulate results from a `get()` call.

6. Looping Through Data

Since Maps store pairs, you have three distinct options for looping based on what data you need.

Option A: Looping through Keys

```
for (String key : capitalCities.keySet()) {  
    System.out.println("Country: " + key);  
}
```

Option B: Looping through Values

```
for (String value : capitalCities.values()) {  
    System.out.println("Capital: " + value);  
}
```

7. Mapping Keys to Values

The most common loop combines both techniques to print the relationships between keys and values.

```
for (String key : capitalCities.keySet()) {  
    System.out.println("Key: " + key + " | Value: " + capitalCities.get(key));  
}
```

Example with Logic (Person/Age)

You can use any Object as a value, including Integers.

```
HashMap<String, Integer> people = new HashMap<>();  
people.put("John", 32);  
people.put("Steve", 30);  
people.put("Angie", 33);
```


8. The 'var' Keyword

From Java 10 onwards, you can simplify your code significantly when declaring local variables. The compiler looks at the right side of the equals sign to determine the type.

```
// Traditional
HashMap<String, String> map = new HashMap<>();

// Modern (Java 10+)
var capitalCities = new HashMap<String, String>();
```

Note: When using `var`, you must include the types in the constructor `new HashMap<String, String>()` so the compiler knows the generic types of the map.

9. Summary & Comparison

Choosing the right collection is vital for application performance.

Feature	List	Set	Map
Duplicates?	Yes	No	Values: Yes / Keys: No
Entry Type	Single Val	Single Val	Key-Value Pair
Maintains Order?	Always	No*	No*

Executive Summary

A HashMap stores items in key/value pairs.

© 2026 CodeHive Academy | Documentation Issue v1.02